

# Jeu 2 : Platformer

---

Un personnage saute de plateforme en plateforme pour ramasser dix objets. Quand tout est ramassé, les objets reviennent et une bombe apparait. Une bombe touchée met fin à la partie. Le score augmente de 10 à chaque objet. Le personnage est une seule image, sans animation.

C'est le jeu du CM (étapes 1 à 6) et du TD1, avec les plateformes placées autrement : tout tient dans un seul fichier JavaScript, avec les fonctions `preload`, `create` et `update`.

Ce guide se suit dans l'ordre. Après chaque étape il y a un test : si tu ne vois pas ce qui est annoncé, ne passe pas à la suite.

## Étape 0 : préparer le thème et les fichiers

---

### Les images du thème

Pour un nouveau thème (dauphin, Noël, rocher, voiture, lunettes...), seules ces images changent. Garde le même nom de fichier.

| Fichier                     | Taille              | À quoi ça sert                                              | Idée                                                |
|-----------------------------|---------------------|-------------------------------------------------------------|-----------------------------------------------------|
| <code>fond.png</code>       | au moins 800 x 600  | décor                                                       | paysage lié au mot                                  |
| <code>plateforme.png</code> | exactement 400 x 32 | le sol (affiché en double largeur) et les trois plateformes | herbe, glace, nuage, pierre                         |
| <code>joueur.png</code>     | environ 32 x 48     | le personnage, une seule image                              | personnage ou objet du thème, tourné vers la droite |
| <code>bon.png</code>        | environ 40 x 40     | objet à ramasser, 10 points                                 | un élément du thème                                 |
| <code>mauvais.png</code>    | environ 30 x 30     | la bombe                                                    | un intrus du thème                                  |

Pour les trouver :

1. Recherche sur internet « mot du thème png transparent » (le fond transparent est important pour `bon`, `mauvais` et `joueur`).
2. Enregistre l'image au format PNG.
3. Redimensionne-la à la taille du tableau avec un site de redimensionnement d'image ou avec Paint (Redimensionner, en pixels, sans conserver les proportions si besoin).
4. Renomme-la avec le nom du tableau.

### Le joueur : une seule image

Le joueur est une seule image, sans animation. Trois conseils :

- Taille : environ 32 x 48 pixels. Phaser utilise la taille de l'image comme zone de collision : une image trop grande donne un joueur énorme qui ne passe plus entre les plateformes.

- Orientation : l'image doit regarder vers la droite. Quand le joueur va à gauche, le code retourne l'image avec `flipX` (TD3).
- Fond transparent, comme pour les autres images.

## Les fichiers à récupérer sur le réseau

- `phaser.min.js` : dans le dossier `js` du TD3 (version 3.55). Le CM utilise un lien vers internet, mais une copie locale évite les soucis.
- Les sons du TD1 : `ding.mp3` devient `bon.mp3`, `criMort.mp3` devient `mauvais.mp3`.

## Les dossiers

Dans ton W, crée cette arborescence :



## Lancer le jeu

Ouvre toujours le jeu avec l'adresse du serveur web de l'IUT, pas en double-cliquant sur `index.html`. Sinon les images ne se chargent pas (CM : « utiliser un serveur web »). Après chaque modification, enregistre et recharge la page (F5 ou Ctrl + F5).

Pour voir les erreurs : touche F12, onglet Console.

## Étape 1 : `index.html`

C'est le fichier passe-partout du CM, avec Phaser puis ton script.

```

<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Platformer</title>
  <script src="dist/js/lib/phaser.min.js"></script>
  <style type="text/css">
    body {
      margin: 0;
    }
  </style>
</head>
<body>
  <script src="dist/js/game.js"></script>
</body>
</html>

```

## Étape 2 : la config et le jeu (début de `game.js`)

La config donne la taille de la fenêtre et la physique Arcade avec une gravité de 300 (CM, étapes 1 et 3). `scene` indique les trois fonctions du jeu. La clé `key: 'jeu'` sert plus tard à relancer la partie.

```

var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: { y: 300 },
      debug: false
    }
  },
  scene: {
    key: 'jeu',
    preload: preload,
    create: create,
    update: update
  }
};

var game = new Phaser.Game(config);

```

Ensuite, déclare toutes les variables du jeu. Elles servent dans plusieurs fonctions, on les utilisera au fil des étapes.

```

var platforms;
var player;
var cursors;
var stars;
var bombs;
var score;
var scoreText;
var gameOver;
var sonBon;
var sonMauvais;

```

Ajoute aussi trois fonctions vides, sinon Phaser ne trouve rien à appeler :

```
function preload() {  
}  
  
function create() {  
}  
  
function update() {  
}
```

**Test.** Un rectangle noir de 800 x 600 apparaît. La console ne montre aucune erreur rouge.

## Étape 3 : charger les ressources (preload)

CM, étape 2. Le premier nom est la clé qu'on utilisera ensuite, le second le chemin du fichier.

```
function preload() {  
    this.load.image('fond', 'dist/assets/img/fond.png');  
    this.load.image('plateforme', 'dist/assets/img/plateforme.png');  
    this.load.image('bon', 'dist/assets/img/bon.png');  
    this.load.image('mauvais', 'dist/assets/img/mauvais.png');  
    this.load.image('joueur', 'dist/assets/img/joueur.png');  
}
```

Remplace la fonction `preload` vide par celle-ci.

## Étape 4 : le monde (create)

CM, étape 3. Les plateformes sont dans un groupe statique : elles ne bougent pas et la gravité ne les touche pas.

```
function create() {  
    // valeurs de départ (create est appelé à chaque nouvelle partie)  
    score = 0;  
    gameOver = false;  
  
    this.add.image(400, 300, 'fond');  
  
    // le sol et les plateformes  
    platforms = this.physics.add.staticGroup();  
    platforms.create(400, 568, 'plateforme').setScale(2).refreshBody();  
    platforms.create(650, 450, 'plateforme');  
    platforms.create(150, 340, 'plateforme');  
    platforms.create(600, 230, 'plateforme');  
    platforms.create(100, 120, 'plateforme');  
}
```

Le sol utilise la même image, agrandie deux fois pour couvrir toute la largeur. Les quatre autres plateformes forment un zigzag : une à droite, une à gauche, une à droite, une à gauche, de plus en plus haut.

| Plateforme | x   | y   |
|------------|-----|-----|
| sol        | 400 | 568 |
| 1          | 650 | 450 |
| 2          | 150 | 340 |
| 3          | 600 | 230 |
| 4          | 100 | 120 |

`x` et `y` sont les coordonnées du centre de l'image. Pour déplacer une plateforme, change ces deux nombres. Entre deux plateformes, ne dépasse pas environ 180 pixels de hauteur, sinon le saut ne suffit plus.

**Test.** Le fond, le sol et trois plateformes sont visibles.

## Étape 5 : le joueur

CM, étape 4. Dans `create`, à la suite de ce qui existe :

```
// le joueur
player = this.physics.add.sprite(400, 450, 'joueur');
player.setBounce(0.2);
player.setCollideWorldBounds(true);
this.physics.add.collider(player, platforms);
```

Le joueur est un sprite à physique dynamique : la gravité s'applique, il rebondit un peu (`setBounce`) et ne sort pas de l'écran (`setCollideWorldBounds`). Le `collider` l'empêche de traverser les plateformes.

Le CM ajoute aussi `player.body.setGravityY(300)`. On ne le met pas ici : avec cette gravité en plus de celle de la config, le saut n'est pas assez haut pour atteindre les plateformes.

**Test.** Le joueur tombe de son point de départ et se pose sur le sol. Il rebondit un tout petit peu.

## Étape 6 : le clavier

CM, étape 5. Dans `create`, à la suite :

```
cursors = this.input.keyboard.createCursorKeys();
```

Remplace la fonction `update` vide. Elle est appelée environ 60 fois par seconde. Le saut n'est possible que si le joueur touche le sol (`touching.down`), sinon il pourrait sauter en l'air.

```

function update() {
    if (gameOver) {
        return;
    }

    if (cursors.left.isDown) {
        player.setVelocityX(-160);
        player.flipX = true;
    } else if (cursors.right.isDown) {
        player.setVelocityX(160);
        player.flipX = false;
    } else {
        player.setVelocityX(0);
    }

    // saut : seulement si le joueur touche le sol
    if (cursors.up.isDown && player.body.touching.down) {
        player.setVelocityY(-350);
    }
}

```

Le `if (gameOver)` du début arrête le contrôle du joueur quand la partie est finie. `gameOver` est une variable déclarée à l'étape 2 et mise à `false` dans `create`. `flipX = true` retourne l'image pour regarder à gauche, `false` la remet à droite.

**Test.** Flèches gauche et droite pour avancer, flèche haut pour sauter. Le joueur peut monter sur la première plateforme, puis sur les autres, jusqu'en haut.

## Étape 7 : les objets à ramasser

CM, étape 6. Dans `create`, à la suite. Le groupe contient 10 objets espacés de 80 pixels. Chacun a un rebond différent, entre 0,4 et 0,8.

```

// les objets à récupérer : 10 objets espacés de 80 pixels
stars = this.physics.add.group({
    key: 'bon',
    repeat: 9,
    setXY: { x: 40, y: 0, stepX: 80 }
});

stars.children.iterate(function (child) {
    child.setBounceY(Phaser.Math.FloatBetween(0.4, 0.8));
});

this.physics.add.collider(stars, platforms);
this.physics.add.overlap(player, stars, collectStar, null, this);

```

Le `overlap` appelle la fonction `collectStar` quand le joueur touche un objet. Écris cette fonction à la suite du fichier, en dehors des autres :

```

function collectStar(player, star) {
    star.disableBody(true, true);
}

```

**Test.** Dix objets tombent et rebondissent sur les plateformes. Le joueur les fait disparaître en les touchant.

## Étape 8 : le score

---

TD1, partie 3. Dans `create`, à la suite :

```
scoreText = this.add.text(16, 16, 'Score : 0', { fontFamily: 'Courier New', fontSize: '32px', fill: '#000' });
```

Complète `collectStar` pour compter les points :

```
function collectStar(player, star) {
    star.disableBody(true, true);

    score += 10;
    scoreText.setText('Score : ' + score);
}
```

**Test.** Le score monte de 10 à chaque objet.

## Étape 9 : un nouveau tour et des bombes

---

TD1, partie 3. Dans `create`, à la suite, on crée le groupe de bombes. Elles rebondissent sur les plateformes, et le joueur y est sensible avec `hitBomb`.

```
// les objets à éviter
bombs = this.physics.add.group();
this.physics.add.collider(bombs, platforms);
this.physics.add.collider(player, bombs, hitBomb, null, this);
```

Dans `collectStar`, à la fin, ajoute le nouveau tour : quand il n'y a plus d'objet actif, on les remet tous en haut et une bombe apparaît.

```
// tout est ramassé : nouveau tour et une bombe de plus
if (stars.countActive(true) === 0) {
    stars.children.iterate(function (child) {
        child.enableBody(true, child.x, 0, true, true);
    });

    // la bombe apparaît du côté opposé au joueur
    var x = (player.x < 400) ? Phaser.Math.FloatBetween(400, 800) :
    Phaser.Math.FloatBetween(0, 400);

    var bomb = bombs.create(x, 16, 'mauvais');
    bomb.setBounce(1);
    bomb.setCollideWorldBounds(true);
    bomb.setVelocity(Phaser.Math.FloatBetween(-200, 200), 20);
    bomb.body.allowGravity = false;
}
```

`bomb.setBounce(1)` fait rebondir la bombe sans jamais perdre de vitesse. Le TD écrit `bomb.allowGravity = false;`, mais c'est `bomb.body.allowGravity` qui supprime vraiment la gravité.

Il manque encore la fonction `hitBomb`, qu'on écrit à l'étape suivante. Pour tester maintenant, ajoute-la vide à la fin du fichier :

```
function hitBomb(player, bomb) {  
}
```

**Test.** Après le 10e objet, les objets reviennent en haut et une bombe se met à rebondir partout. Une bombe de plus à chaque tour.

## Étape 10 : la fin de partie

TD1, partie 3. Remplace la fonction `hitBomb` vide. Le texte « Rejouer » est un bouton : au clic, `this.scene.start('jeu')` relance la scène, avec la clé donnée dans la config (TD3, partie 4).

```
function hitBomb(player, bomb) {  
    this.physics.pause();           // physique en pause  
    player.setTint(0xff0000);        // le joueur devient rouge  
    gameOver = true;                 // on arrête le jeu à l'aide d'un boolean  
  
    this.add.text(400, 230, 'Perdu', { fontFamily: 'Georgia', fontSize: '80px', fill:  
    '#000' })  
        .setOrigin(0.5);  
  
    this.add.text(400, 330, 'Rejouer', { fontFamily: 'Georgia', fontSize: '48px',  
    fill: '#fff', backgroundColor: '#000' })  
        .setOrigin(0.5)  
        .setInteractive()  
        .on('pointerdown', () => {  
            this.scene.start('jeu');  
        });  
}
```

**Test.** Toucher une bombe arrête tout, le joueur devient rouge et « Perdu » apparaît. Un clic sur « Rejouer » relance une partie à zéro, score compris.

## Étape 11 : les sons

TD1, partie 3. Dans `preload`, ajoute :

```
this.load.audio('sonBon', 'dist/assets/sounds/bon.mp3');  
this.load.audio('sonMauvais', 'dist/assets/sounds/mauvais.mp3');
```

À la fin de `create` :

```
// sons  
sonBon = this.sound.add('sonBon');  
sonMauvais = this.sound.add('sonMauvais');
```

Dans `collectStar`, juste après `star.disableBody(true, true);` :

```
sonBon.play();
```

Dans `hitBomb`, juste après `gameOver = true;` :

```
sonMauvais.play();
```

**Test final.** Un son à chaque objet, un autre à la bombe. Si tu n'entends rien, clique d'abord dans la page : certains navigateurs bloquent le son avant le premier clic.

## Problèmes fréquents

| Ce qu'on voit                                       | Cause probable                                                                   |
|-----------------------------------------------------|----------------------------------------------------------------------------------|
| Page vide                                           | jeu ouvert par le fichier et pas par le serveur ; erreur dans la console (F12)   |
| <code>Phaser is not defined</code>                  | <code>phaser.min.js</code> absent ou chargé après <code>game.js</code>           |
| Une image est remplacée par un carré avec une croix | mauvais nom ou mauvais chemin dans <code>preload</code> ; majuscules différentes |
| Le joueur traverse le sol                           | collider du joueur oublié                                                        |
| Le joueur ne saute pas                              | la condition <code>touching.down</code> ou <code>cursors.up</code> mal écrite    |
| Impossible d'atteindre une plateforme               | <code>plateforme.png</code> n'a pas la bonne taille, ou saut changé              |
| Les objets ne rebondissent pas sur les plateformes  | collider des objets oublié                                                       |
| <code>collectStar is not defined</code>             | la fonction n'est pas écrite, ou elle est dans une autre fonction                |
| Rejouer ne remet pas le score à zéro                | <code>score = 0;</code> absent au début de <code>create</code>                   |

## Pour aller plus loin

- Le bouton plein écran (TD1).
- Une musique en boucle (TD3).
- Plusieurs vies avant la fin de partie.

## Annexe : code complet

Le code final de chaque fichier, pour comparer avec le tien.

## index.html

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Platformer</title>
  <script src="dist/js/lib/phaser.min.js"></script>
  <style type="text/css">
    body {
      margin: 0;
    }
  </style>
</head>
<body>
  <script src="dist/js/game.js"></script>
</body>
</html>
```

```

var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  physics: {
    default: 'arcade',
    arcade: {
      gravity: { y: 300 },
      debug: false
    }
  },
  scene: {
    key: 'jeu',
    preload: preload,
    create: create,
    update: update
  }
};

var game = new Phaser.Game(config);

var platforms;
var player;
var cursors;
var stars;
var bombs;
var score;
var scoreText;
var gameOver;
var sonBon;
var sonMauvais;

function preload() {
  this.load.image('fond', 'dist/assets/img/fond.png');
  this.load.image('plateforme', 'dist/assets/img/plateforme.png');
  this.load.image('bon', 'dist/assets/img/bon.png');
  this.load.image('mauvais', 'dist/assets/img/mauvais.png');
  this.load.image('joueur', 'dist/assets/img/joueur.png');

  this.load.audio('sonBon', 'dist/assets/sounds/bon.mp3');
  this.load.audio('sonMauvais', 'dist/assets/sounds/mauvais.mp3');
}

function create() {
  // valeurs de départ (create est appelé à chaque nouvelle partie)
  score = 0;
  gameOver = false;

  this.add.image(400, 300, 'fond');

  // le sol et les plateformes
  platforms = this.physics.add.staticGroup();
  platforms.create(400, 568, 'plateforme').setScale(2).refreshBody();
  platforms.create(650, 450, 'plateforme');
  platforms.create(150, 340, 'plateforme');
  platforms.create(600, 230, 'plateforme');
  platforms.create(100, 120, 'plateforme');

```

```

// le joueur
player = this.physics.add.sprite(400, 450, 'joueur');
player.setBounce(0.2);
player.setCollideWorldBounds(true);
this.physics.add.collider(player, platforms);

cursors = this.input.keyboard.createCursorKeys();

// les objets à récupérer : 10 objets espacés de 80 pixels
stars = this.physics.add.group({
    key: 'bon',
    repeat: 9,
    setXY: { x: 40, y: 0, stepX: 80 }
});

stars.children.iterate(function (child) {
    child.setBounceY(Phaser.Math.FloatBetween(0.4, 0.8));
});

this.physics.add.collider(stars, platforms);
this.physics.add.overlap(player, stars, collectStar, null, this);

// les objets à éviter
bombs = this.physics.add.group();
this.physics.add.collider(bombs, platforms);
this.physics.add.collider(player, bombs, hitBomb, null, this);

scoreText = this.add.text(16, 16, 'Score : 0', { fontFamily: 'Courier New',
fontSize: '32px', fill: '#000' });

// sons
sonBon = this.sound.add('sonBon');
sonMauvais = this.sound.add('sonMauvais');
}

function update() {
    if (gameOver) {
        return;
    }

    if (cursors.left.isDown) {
        player.setVelocityX(-160);
        player.flipX = true;
    } else if (cursors.right.isDown) {
        player.setVelocityX(160);
        player.flipX = false;
    } else {
        player.setVelocityX(0);
    }

    // saut : seulement si le joueur touche le sol
    if (cursors.up.isDown && player.body.touching.down) {
        player.setVelocityY(-350);
    }
}

function collectStar(player, star) {
    star.disableBody(true, true);
    sonBon.play();
}

```

```

score += 10;
scoreText.setText('Score : ' + score);

// tout est ramassé : nouveau tour et une bombe de plus
if (stars.countActive(true) === 0) {
    stars.children.iterate(function (child) {
        child.enableBody(true, child.x, 0, true, true);
    });

    // la bombe apparait du côté opposé au joueur
    var x = (player.x < 400) ? Phaser.Math.FloatBetween(400, 800) :
Phaser.Math.FloatBetween(0, 400);

    var bomb = bombs.create(x, 16, 'mauvais');
    bomb.setBounce(1);
    bomb.setCollideWorldBounds(true);
    bomb.setVelocity(Phaser.Math.FloatBetween(-200, 200), 20);
    bomb.body.allowGravity = false;
}

function hitBomb(player, bomb) {
    this.physics.pause(); // physique en pause
    player.setTint(0xff0000); // le joueur devient rouge
    gameOver = true; // on arrête le jeu à l'aide d'un boolean
    sonMauvais.play();

    this.add.text(400, 230, 'Perdu', { fontFamily: 'Georgia', fontSize: '80px', fill:
'#000' })
        .setOrigin(0.5);

    this.add.text(400, 330, 'Rejouer', { fontFamily: 'Georgia', fontSize: '48px',
fill: '#fff', backgroundColor: '#000' })
        .setOrigin(0.5)
        .setInteractive()
        .on('pointerdown', () => {
            this.scene.start('jeu');
        });
}

```